Search: Suche mit Breitensuche

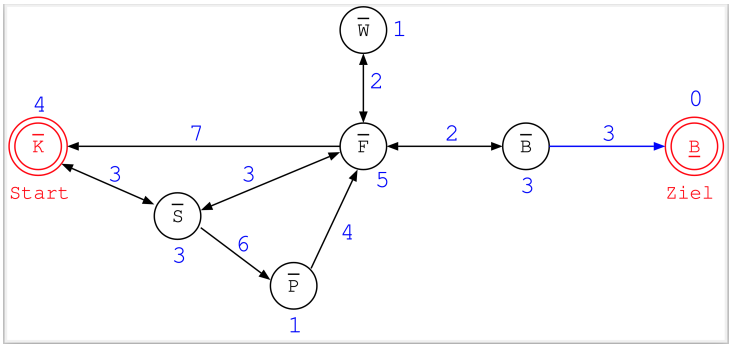
Carsten Gips (HSBI)

TL;DR
Die Breitensuche gehört zu den “Uninformierten Suchverfahren”: Es werden keine weiteren Pfadkosten, sondern nur die Anzahl der Schritte berücksichtigt.
Die Breitensuche entsteht, wenn man bei der Tree-Search oder der Graph-Search für die Datenstruktur eine **Queue** benutzt: Expandierte Nachfolger werden immer **hinten** in die Queue eingefügt, und der nächste zu expandierende Knoten wird **vorn** aus der Queue genommen. Dadurch wird bei der Breitensuche der Suchbaum ebenenweise entwickelt.

Videos

- VL Breitensuche

1 Hole das Buch



=> **Problemlösen == Suche im Graphen**

Uninformierte (“blinde”) Suche:

Keine Informationen über die Kosten eines Pfades: Nur die Pfadlänge (Anzahl der Schritte) zählt.

Varianten:

- Tiefensuche
- **Breitensuche**

2 Breitensuche (*BS*, *BFS*)

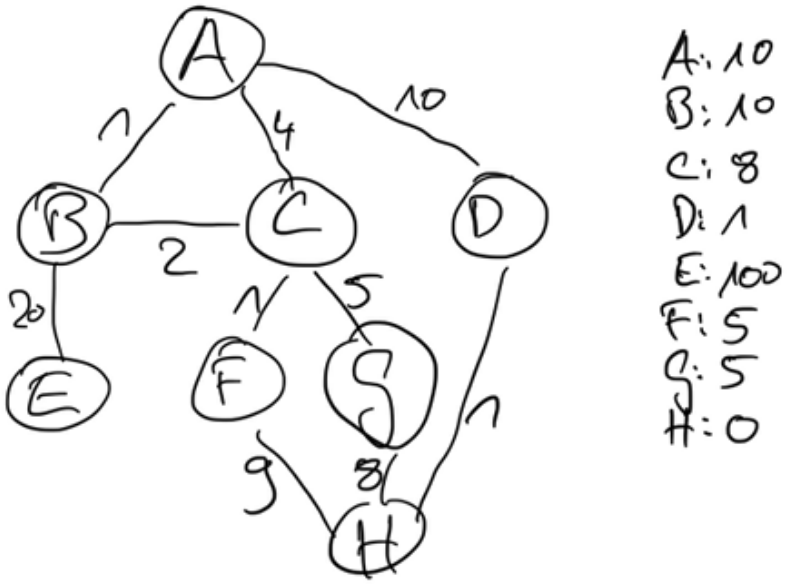
Erinnerung Graph-Search

1. Füge Startknoten in leere Datenstruktur (Stack, Queue, …) ein
2. Entnehme Knoten aus der Datenstruktur:
    - Knoten ist gesuchtes Element: Abbruch, melde “*gefunden*”

- Markiere aktuellen Knoten, und
- Expandiere alle Nachfolger des Knotens und füge alle unmarkierten Nachfolger, die noch nicht in der Datenstruktur sind, in die Datenstruktur ein

3. Falls die Datenstruktur leer ist: Abbruch, melde "*nicht gefunden*"
4. Gehe zu Schritt 2

=> Was passiert, wenn wir eine **Queue** einsetzen?



3 Bemerkungen

- Nachfolger eines Knotens: Alle von diesem Zustand durch Aktionen erreichbare Zustände
- Suchalgorithmus mit **Queue** als Datenstruktur => **Breitensuche**
  - Zu betrachtender Knoten in Schritt 2 wird *vorn* aus der Queue genommen
  - Expandierte Knoten werden in Schritt 2.a *hinten* in die Queue eingefügt Dabei i.A. die vorgegebene Reihenfolge der Nachfolgeknoten beachten!

  Auswirkung: Suchbaum wird **ebenenweise** aufgebaut (deshalb "Breitensuche")
- Graph-Search: Markierte Knoten müssen geeignet gespeichert werden: separate Datenstruktur => Aufwand!

4 Konventionen für diese Lehrveranstaltung

In der Beschreibung der Algorithmen werden häufig nur die letzten Knoten der partiellen Wege in den Datenstrukturen mitgeführt (das gilt auch für die Beschreibung im (Russell und Norvig 2021)). Dies erschwert die

Nachvollziehbarkeit, wenn man die Queue oder den Stack schrittweise aufschreibt. Deshalb wird für diese Veranstaltung die Konvention eingeführt, immer die **partiellen Wege** aufzuschreiben.

Nicht Bestandteil der Algorithmen, dient aber der Nachvollziehbarkeit: Expandierte Knoten sollen alphabetisch sortiert an der korrekten Stelle in der Datenstruktur auftauchen, dabei soll aber natürlich die Reihenfolge der ursprünglich in der Datenstruktur enthaltenen Knoten nicht modifiziert werden. (Bei "echten" Problemen wird die Reihenfolge der expandierten Nachfolger in der Regel durch eine Reihenfolge der anwendbaren Operationen bestimmt.)

5 Weitere Hinweise

- Die Breitensuche wurde zufällig am Beispiel Graph-Search eingeführt. Man kann auch die Tree-Search-Variante einsetzen. Wichtig ist nur, dass als Datenstruktur eine Queue genutzt wird.
- Im (Russell und Norvig 2021) wird die Breitensuche ebenfalls auf der Basis des Graph-Search-Algorithmus eingeführt. Allerdings wird die Abbruchbedingung modifiziert: Die Zielbedingung wird nicht erst (wie bei Graph-Search eigentlich definiert) geprüft, wenn ein Knoten aus der Queue entnommen wird, sondern bereits bei der Erzeugung der Nachfolgerknoten (vor dem Einfügen in die Queue). Dadurch spart man sich die Expansion einer zusätzlichen Ebene: Die Komplexität wäre in diesem Fall "nur" $O(b^d)$.

6 Eigenschaften Breitensuche vs. Tiefensuche

	Tiefensuche	**Breitensuche**
Vollständigkeit	nein[1]	ja[2]
Optimalität	nein	ja
Zeitkomplexität	$O(b^m)$	$O(b^{d+1})$
Speicherkomplexität	$O(bm)$	$O(b^{d+1})$

- Zeitkomplexität: maximal zu expandierende Knotenzahl
- Speicher:
  - TS: in jeder Tiefe weitere b Knoten speichern
  - BS: alle Knoten einer Ebene im Speicher halten[3]

b: Verzweigungsfaktor, **d**: Ebene d. höchsten Lösungsknotens, **m**: Länge d. längsten Pfades

[1] gilt für Tree-Search-Variante; vollständig in Graph-Search-Variante bei endlichem Suchraum
[2] falls *b* endlich
[3] $O(b^d)$ mit vorgezogener Zielprüfung (vgl. (Russell und Norvig 2021))

7 Praxisvergleich Breitensuche vs. Tiefensuche

Breitensuche: Annahme: $b = 10$, 10.000 Knoten/s, 1.000 Byte/Knoten

Tiefe	exp. Knoten	Zeit	Speicher
2	10^3	0.1 s	1 MB
4	10^5	10 s	100 MB
6	10^7	20 min	10 GB
8	10^9	30 h	1 TB
10	10^{11}	130 d	100 TB

Tiefensuche: Annahme: längster Pfad (Tiefe) $m = 1000$

=> Speicherbedarf **ca. 10 MB**

8 Wrap-Up

- Uninformierte Suchverfahren
  - Keine weiteren Pfadkosten (nur Anzahl der Schritte)
  - Breitensuche: Verfolge alle Pfade (baue den Suchbaum ebenenweise auf)

9 Zum Nachlesen

- Russell und Norvig (2021): Breitensuche: Abschnitt 3.4.1

Lernziele

- k2: Ich kann erklären, welche Datenstrukturen in der Breitensuche verwendet werden und wie diese sich auswirken
- k2: Ich kann die algorithmische Abläufe und die Terminierung der Breitensuche erklären
- k2: Ich kann Optimalität, Vollständigkeit und Komplexität für die Breitensuche erklären
- k3: Ich kann die Breitensuche implementieren und auf ein konkretes Beispiel anwenden

Quizzes

- Selbsttest Breitensuche (ILIAS)

Quellen

Russell, S., und P. Norvig. 2021. *Artificial Intelligence: A Modern Approach*. 4th Edition. Pearson. http://aima.cs.berkeley.edu.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

> **Last modified:** f393f68 (lecture: fix publishing year of AIMA, 2025-09-27)